CHOREO

算子专用编程语言介绍系列

编程篇之一：加一点儿动态性

Choreo 中的动态形状支持

动态形状在自然语言处理、音频/视频处理的场景下都可能产生。这对算子的编写造成了很大的挑战。因为优化的算子往往需要配合一定的形状输入才能正常工作工作。而动态形状就需要算子对更多的输入形状进行适配。

这一次，让我们使用 Choreo 的动态形状特性来编写一个矩阵乘法吧。这可能是你见过的**最简单**的包含动态形状支持的代码。

```
__co__ auto matmul(s32 [M, K] lhs, s32 [K, N] rhs) {
 s32 [M, N] output;

 // set the L1 block dimensions
 block_m = 4, block_k = 256, block_n = 64;

 parallel p by 1
  with {m_tile, k_tile, n_tile}
   in [M/block_m, K/block_k, N/block_n] // the tiling factors

   foreach m_tile, n_tile {
     // define the L1 buffer and initialize
     local s32 [block_m, block_n] l1_out {0};

     foreach k_tile {
       lf = dma.copy.async lhs.chunkat(m_tile, k_tile) => local;
       rf = dma.copy.async rhs.chunkat(k_tile, n_tile) => local;
       wait lf , rf;

       // calculating in a single block
       foreach {m, n, k} in [block_m, block_n, block_k]
         l1_out.at(m, n) +=
            lf.data.at(m, k) * rf.data.at(k, n);

       dma.copy l1_out => output.chunkat(m_tile, n_tile);
     }
   }

 return output;
}
```

可以看到，在这个名为 `matmul` 的 *Tileflow* 的函数中，我们声明张量参数`lhs` 与 `rhs` 的形状分别为 `[M, K]` 与 `[K, N]`。这里的 `M, K, N` 可以是任何运行时期的值，因此参数 `lhs` 与 `rhs` 的形状也通常被被称为**动态形状**。可以看到，在 Choreo 中定义动态形状非常自然，只需要将**形状中的维度符号化**为“变量”即可。

使用这个语言特性，我们所编写的 `matmul` *Tileflow* 函数可以支持维度能被 `4, 256, 64` 整除的多种张量输入数据的组合。Choreo 会验证 `lhs` 的维度 1 与 `rhs` 的维度 0 是否相等。**如果不等，或者出现了张量切分后存在 0 值维度的输入，Choreo 都会及时退出并报错提醒**。

Tileflow 代码逐行解析

- **函数定义**：`matmul` 是一个 Choreo *Tileflow* 函数，它接受两个**动态形状**的二维张量 `lhs` 和 `rhs` 作为输入，返回值则需要编译器推导得出。注意，这里两个输入形状有一个维度 `K` 是等同的。
- **数据分块 (Tiling) 设定**：`block_m/n/k` 设定了分块中的维度大小，而 `with {m/n/t_tile} in ...` 语句则设定了切分值。这里由于 GCU L1 存储的大小固定，我们采用了**分块大小固定，切分值随着输入大小变化而变化**的策略。
- **多层次的迭代**：这里存在三个层次的`foreach` 语句。最外层是关于 `m_tile, n_tile` 的两层循环，该层次中我们在每次迭代都 **初始化本地缓存**，其大小为 `block_m * block_n`。而第二层关于 `m_tile` 的单层循环中，我们使用 **异步数据传输**，将左右操作数分别切分为固定大小，并从从全局内存异步传输到本地内存。第三层次则是个三层 **本地计算** 循环，其将本地的左右操作数依序相乘，累加放入之前初始化的缓存中。
- **结果同步写回**：`dma.copy` 语句将计算结果从本地内存写回到全局内存中的分块位置中。
- **返回结果**：最终返回计算完成的 `output` 数组。

CHOREO

查验数据形状

Choreo 编译器能对数据形状做准确的静态推导。如果在编码时想确切知道推导后的形状，程序员可以使用 Choreo 的 -i选项进行查验。

```
choreo -i matmul.co
```

以上页的 matmul 代码为例，我们可以通过该选项查得 lf 对应的数据块大小为 [4, 256]，而 rf 对应的数据块大小为 [256, 64] ——确实是固定的大小，与设计预期相符。如果你已经编写了自己的 Choreo 程序，那赶紧来试试看吧。

欢迎加入 Choreo 服务群，汇报 Choreo 的问题，分享试用体验，以及提出建议

结合 Choreo 与 Topscc

如果你觉得 Choreo 有可取之处，但遗憾的是不能够重用已有的代码，那这里有一个好消息：Choreo 可以与 Topscc 代码融合。

```
__device__ void kernel(int * a, int * b, int * c, int n) {
  for (int i = 0; i < n; ++i) c[i] = a[i] + b[i];
}

__co__ auto ele_add(s32 [6, 17, 64] lhs, s32 [6, 17, 64] rhs) {
  s32 [lhs.span] output;

  parallel p by 6
    foreach index in [17, 4] {
      lhs_l1 = dma.copy.async lhs.chunkat(p, x, y) => local;
      rhs_l1 = dma.copy.async rhs.chunkat(p, x, y) => local;
      wait lhs_l1, rhs_l1;

      local s32 [lhs_l1.span] l1_out;
      call kernel(lhs_l1.data, rhs_l1.data, l1_out, |lhs_l1.span|);

      out_store = dma.copy.async l1_out => output.chunkat(p, x, y);
      wait out_store;
    }

  return output;
}
```

喏，如你所见，我们定义了一个名为 kernel 的设备函数，它的语法与 Topscc 一模一样——因为这就是一个 Topscc 的函数，你可以在 Choreo 的 *Tileflow* 函数 ele_add 中使用关键字 call 来调用它。而无论你想在设备函数中使用 GCU 的内置函数，或是使用 TCLE 等高级编程接口等等，Choreo 都悉听尊便，并不干涉。

所以，逐步迁移/升级你的代码为 Choreo-C++ 并不困难，因为 Choreo 是一种嵌入式的领域特定语言（Embedded Domain Specific Language，EDSL），而 Topscc 正是 Choreo 的一个宿主！

Choreo Is What You Need

深入了解 Choreo

如果想深入了解 Choreo，我们准备了相关的文档、视频教程供您学习：

- 网页教程：http://choreo.enflame.cn/
- 视频教程：https://learning.enflame-tech.com/course/144

参与到 Choreo 的开发中

无论是汇报问题，或是建议新的 Choreo 特性，都可以通过在 Choreo 项目主页创建 issue 的方式达成：http://git.enflame.cn/xiaofeng.guan/choreo/-/issues

Choreo 的**项目主页**在：http://git.enflame.cn/xiaofeng.guan/choreo，其包含了编译使用 Choreo 的说明

如有需要请联系：

- 官孝峰：xiaofeng.guan@enflame-tech.com
- 石恒：heng.shi@enflame-tech.com

CHOREO

- 简化数据搬运编程
- 提高程序安全性
- 支持动态形状